

World models, active inference, boundaries, and forward compatibility

World models from code

The central theoretical claim of COGANT is that source code implicitly defines a generative model of system behaviour, a claim that is structurally analogous to the *world-model* line of reinforcement-learning work exemplified by Dreamer V3 [hafner2023dreamerv3]: an encoder maps observations to a latent state, latent dynamics predict forward, and a decoder maps back. The Dreamer architecture learns the world model from observation trajectories. COGANT extracts it symbolically from the program graph; the comparison is productive precisely because it clarifies that COGANT produces an explicit, interpretable state-space and transition structure where Dreamer produces an opaque learned latent. Both are generative models that a downstream Active Inference agent [friston2010free; parr2022active; dacosta2020active; heins2022pymdp] can treat as the $p(o, s)$ component of a variational free-energy functional.

Active inference and program behavior

The state-space IR in COGANT's pipeline (states, actions, transitions, observations) shares structural parallels with **active inference** formulations [friston2010free; parr2022active], where an agent maintains beliefs about hidden states and selects actions to minimize prediction error. The discrete-state synthesis presented in [dacosta2020active] is the closest formal target of COGANT's compilation: variables, actions, observation modalities, and transition structures in the Generalized Notation Notation bundle map directly onto the tuples required by a discrete-state active inference agent, and the step-by-step construction protocol of [smith2022stepbystep] can be followed literally against those bundles. PyMDP [heins2022pymdp] provides a reference Python runtime that executes exactly this form of agent, making it a natural downstream consumer of COGANT exports. In the program analysis context, the "agent" is the analysis pipeline itself: it observes code artifacts, maintains beliefs about program behavior (the state-space model), and refines those beliefs as new evidence (dynamic traces, coverage data) arrives.

Boundaries

COGANT does not subsume formal verification, interactive theorem proving, or full interprocedural pointer analysis unless implemented as explicit future stages.

`../cogant/docs/reference/implementation_status.md`
marks Rust acceleration and additional parsers as staged; the manuscript should be read together with that table for up-to-date scope.

Forward compatibility

Promoting COGANT into ../../../../projects/ integrates manuscript PDF rendering with the template's validation gates. Cross-references in this folder use paths **relative to these Markdown files** (for example ../../cogant/docs/) so links stay stable when the tree moves.